

UNIVERSIDAD TECNOLOGICA NACIONAL
Tecnatura Universitaria en Programacion a Distancia

Materia: Programacion 2

TRABAJO PRACTICO INTEGRADOR
"Food Store - Sistema de Gestion de Pedidos de Comida"

Alumna: Eugenia Abril Barco
GRUPO: 77

3. INTRODUCCION

El presente trabajo consiste en el desarrollo de "Food Store", un sistema de consola para la gestion de pedidos de un negocio de comidas, programado en Java aplicando el paradigma de Programacion Orientada a Objetos (POO).

El sistema permite administrar cuatro entidades principales -categorias, productos, usuarios y pedidos- mediante un menu interactivo que ofrece operaciones CRUD (crear, leer, actualizar y eliminar). A diferencia de un sistema con base de datos, toda la informacion se almacena en memoria durante la ejecucion utilizando Colecciones de Java.

El objetivo academico fue integrar y demostrar los conceptos centrales de la materia: herencia, clases abstractas, interfaces, encapsulamiento, polimorfismo, uso de colecciones y manejo de excepciones propias.

4. MARCO TEORICO

Tecnologias utilizadas:

- Java (JDK 21): lenguaje orientado a objetos, multiplataforma, de tipado estatico.
- NetBeans: entorno de desarrollo (IDE) utilizado para escribir, compilar y ejecutar.
- Colecciones (java.util.List / ArrayList): estructuras de datos dinamicas que permiten almacenar y recorrer conjuntos de objetos en memoria.

Conceptos clave de POO aplicados:

- Clase y objeto: una clase es un molde (ej. Producto) y un objeto es una instancia concreta (ej. "Coca Cola").
- Encapsulamiento: los atributos se declaran private y se acceden mediante getters y setters, protegiendo el estado interno del objeto.
- Herencia: las entidades heredan atributos y comportamiento comun de una clase padre (Base), evitando duplicar codigo.
- Clase abstracta: Base no puede instanciarse directamente; sirve unicamente como base de las demas y obliga a implementar metodos (toString).
- Interfaz: Calculable define un "contrato" (el metodo calcularTotal) que la clase Pedido se compromete a implementar.
- Polimorfismo: distintas clases responden al mismo metodo (toString, equals) cada una a su manera; en tiempo de ejecucion Java invoca la version correcta.
- Manejo de excepciones: mediante try/catch y excepciones propias se controlan los errores (datos invalidos, entidad inexistente, falta de stock) evitando que el programa se cierre de forma inesperada.

5. DECISIONES TECNICAS Y ARQUITECTURA

El sistema se organizo en una arquitectura por capas, separando responsabilidades:

- | UI / Menus (paquete ui)
- | - Interactua con el usuario (Scanner).
- | - Muestra menus y mensajes. Captura excepciones.

llama a....

- | Services (paquete services)
- | - Logica de negocio y validaciones.
- | - Guardan las entidades en Colecciones (memoria).
- | - Lanzas excepciones propias ante errores.

Usa....

- | Entities (paquete entities) + enums + interfaces
- | - Modelo de dominio (clases del UML).

Paquetes del proyecto:

- integrado.prog2 -> Main (punto de entrada)
- integrado.prog2.entities -> Base, Categoria, Producto, Usuario, Pedido, DetallePedido
- integrado.prog2.enums -> Rol, Estado, FormaPago
- integrado.prog2.interfaces -> Calculable
- integrado.prog2.exception -> EntidadNoEncontradaException, StockInvalidoException, DatoInvalidoException, MailDuplicadoException
- integrado.prog2.services -> CategoriaService, ProductoService, UsuarioService, PedidoService
- integrado.prog2.ui -> ConsolaUtil + un Menu por cada entidad

Algunas decisiones destacadas:

1. Clase abstracta Base: centraliza id (autogenerado con un contador static), eliminado (para la baja logica) y createdAt. Todas las entidades la heredan.
2. Baja logica (soft delete): eliminar no borra el objeto de la coleccion, sino que marca eliminado = true. Los listados filtran los eliminados, pero el historial se conserva (importante para pedidos ya realizados).
3. Creacion de pedido en pasos (atomica): el pedido se arma en memoria y se cargan sus detalles uno a uno descontando stock. Si un detalle falla (por ejemplo, falta de stock), se captura la excepcion y se cancela el pedido completo reponiendo el stock, de modo de no dejar datos inconsistentes.
4. Calculo del total via interfaz: el total del pedido se obtiene con calcularTotal() de la interfaz Calculable, recorriendo los subtotales de cada detalle.

.6. DIFICULTADES Y SOLUCIONES

- Dificultad: evitar que un pedido quedara a medias si un producto no tenia stock suficiente.
Solucion: se implemento la creacion del pedido en pasos, con reposicion de stock (rollback) cuando se cancela, asegurando consistencia de los datos.
- Dificultad: no repetir codigo de validacion de lectura de teclado en cada menu.

Solucion: se creo la clase ConsolaUtil, que centraliza la lectura segura de numeros, textos y confirmaciones.

- Dificultad: diferenciar la responsabilidad de cada clase.

Solucion: se separo el proyecto en capas (entities, services, ui), dejando la logica de negocio en los services y la interaccion en los menus.

7. CONCLUSION

El TPI permitio integrar todos los conceptos de POO vistos en la materia en un sistema funcional completo, comprendiendo la importancia de la separacion de responsabilidades, las validaciones y el manejo de errores para construir software robusto.

- Repositorio del proyecto (GitHub): https://github.com/mumi-b/TPI_FoodStore_Barco

- Video demostrativo en 4 partes:

1- <https://www.loom.com/share/a6209edec40647b281d63b54841017a5>

2 - <https://www.loom.com/share/4037226db33a44159a42e75c448acb25>

3 - <https://www.loom.com/share/c4136ff46cae462080ae9c390901a939>

4 - <https://www.loom.com/share/b0ac0bdb4ed14d7fb3a3fff3e209253f>

De ante mano pido disculpas por presentar así el video, me quede sin tiempo para rehacerlo de una sola vez, y la aplicación con la que lo hice me exigía pagar una membresía para poder descargarlos y editarlos. Lo puedo volver a hacer mejor, pero quería al menos cumplir con la entrega del trabajo y el video.

Muchas Gracias